CSHL BIOLOGICAL DATA SCIENCE · NOVEMBER 4-7, 2026

From research code to reproducible infrastructure with Galaxy user-defined tools

Marius van den Beek[1], **John Chilton**[1], **Dannon Baker**[2] [1]The Pennsylvania State University · [2]Johns Hopkins University

Galaxy has thousands of tools, yet the one step you need is often missing. A UDT closes that **last mile**: write it, test it on real data on a production server, iterate, then **graduate** it to a community tool.

Galaxy PROJECT

Try it on usegalaxy.org
Request access once at udt-signup.galaxyproject.org. No install or review per tool.

A The gap

Research code starts as a notebook, a lab script, a published method, or AI-drafted code. Writing the first version is the easy part. To become a step you can rerun, scale up, and share, it also needs:

1. **Interface** — typed inputs and outputs
2. **Environment** — the exact software
3. **Data** — large datasets, in place
4. **Compute** — cores, memory, GPUs
5. **Provenance** — what ran, on what, how
6. **Sharing** — a step others can reuse

On a shared Galaxy server, a new tool must also be wrapped, reviewed, and installed by an administrator: the right default for shared tools, but slow for one small step.

A user-defined tool (UDT) covers all six in about 40 lines, in your own account, with no install to wait for.

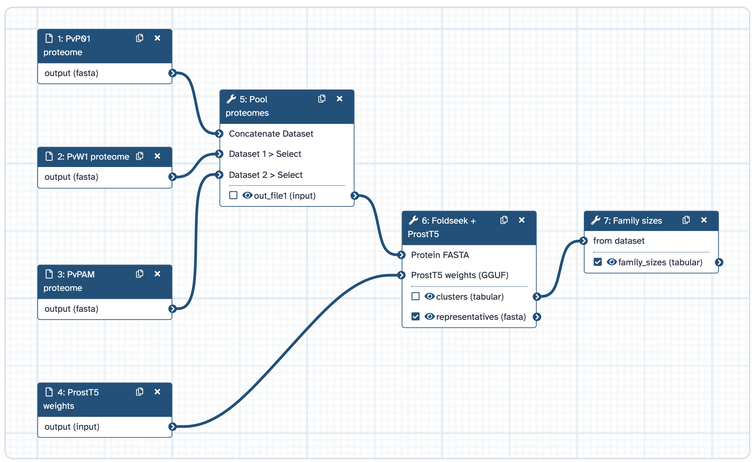
B The last mile

3 Data · 6 Sharing

usegalaxy.org runs **~2,500 tools** from a Tool Shed of **8,000+ repositories**. What's missing is usually one small step: a new method, an extra option, a wrapper merged but not yet installed.

Write → **Run on real data** → **Inspect** → **Edit** ↻

Each round is a save in the tool editor, on the production server with real data.



Three *Plasmodium vivax* proteomes (PvP01, PvW1, PvPAM) are pooled, clustered by the Foldseek UDT (step 6), and counted into family sizes with standard tools. Importing the workflow brings the UDT along.

Then graduate it to a community tool

UDT in your account → **tools-iwc-lab** → **tools-iuc**

When a UDT proves itself, wrap it as a standard tool and open a pull request to **galaxyproject/tools-iwc-lab**. Planemo lints and tests it in CI, then publishes it to the Tool Shed under `iwc-lab`. Mature wrappers graduate to `tools-iuc` for installation on any Galaxy server.

C Anatomy of a UDT

Here in Galaxy's tool editor, a Foldseek UDT clusters ProstT5-predicted structures on usegalaxy.org using 32 cores and a GPU.

Tool Editor

```
1  class: GalaxyUserTool
2  id: foldseek_prostt5_cluster
3  name: Foldseek structural clustering
4  version: 1.0.0
5  description: Cluster proteins by predicted structure on a GPU
6  container: quay.io/biocontainers/foldseek:10.941cd33
7  requirements:
8    - type: resource
9      cores_min: 32
10     ram_min: 8192
11     cuda_device_count_min: 1
12 shell_command: |
13   mkdir weights && cp '$(inputs.model.path)' \
14     weights/prostt5-f16.gguf
15   foldseek easy-cluster '$(inputs.seqs.pa
16     --prostt5-model weights --gpu 1 \
17     --threads "$GALAXY_SLOTS" \
18     -c $(inputs.coverage) -e $(inputs.evalue)
19 inputs:
20   - name: seqs
21     label: Protein FASTA
22     type: data
23     format:
24       - fasta
25   - name: model
26     label: ProstT5 weights (GGUF)
27     type: data
28   - name: coverage
29     label: Minimum alignment coverage
30     type: float
31     value: 0.8
32   - name: evalue
33     label: E-value threshold
34     type: float
35     value: 0.001
36 outputs:
37   - name: clusters
38     type: data
39     format: tabular
40     from_work_dir: clu_cluster.tsv
41   - name: representatives
42     type: data
43     format: fasta
44     from_work_dir: clu_rep_seq.fasta
```

(property) evalue: number
E-value threshold

1. The container is the environment, pinned by tag
2. A resource request: cores, memory, a GPU (**E**)
3. `$(inputs.…)` are typed, sandboxed expressions—hover shows each type (**D**)
4. `$GALAXY_SLOTS`: the site sets the core count
5. Typed inputs become the form in **F**
6. Outputs become datasets with provenance

D Safe by construction

In the tool definition

- Commands are templated with sandboxed JavaScript expressions that cannot reach the database or filesystem
- A container is mandatory
- Inputs are typed and validated before a job exists

At the site

- Access is granted once per account
- Routing and resource limits stay with the site (**E**)

Each job runs with minimal permissions and can only access its own input and output data.

E Requesting compute

The tool declares what it needs

```
cores_min: 32                → cores 32
ram_min: 8192                → mem 8 GiB
cuda_device_count_min: 1     → gpus 1
```

The site's job router (TPV) applies policy

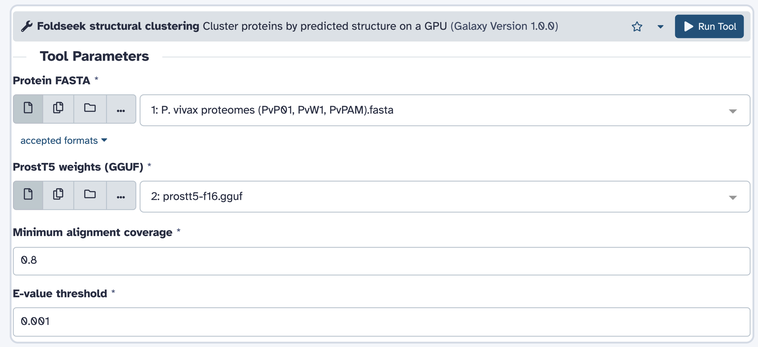
- Only destinations that opt in accept UDT jobs
- GPU requests need the **GPU Access** role
- Per-user pools cap a user's total cores, memory, GPUs

Jetstream2 GPU node (NSF ACCESS)

- Pulsar + Singularity, container required
- 32 cores map to `$GALAXY_SLOTS`, plus 1 GPU

F What Galaxy does with it

Validate → **Build form** → **Run in container** → **Record provenance** → **Reuse in workflows**



Galaxy builds this form directly from the `inputs` block: dataset pickers filtered by format and labeled numeric fields with defaults.

G What AI agents do with UDTs

Galaxy Workflow Foundry — PIPELINE TRANSLATION

Galaxy Workflow Foundry gives AI agents the domain knowledge to translate external pipelines into native Galaxy workflows. When we converted `nf-core/rnaseq` for usegalaxy.org, every step mapped to an installed Galaxy tool except RSEM quantification, whose community wrapper was merged but not yet installed. The agent wrote an RSEM UDT so the workflow could run immediately. Its outputs matched the community wrapper's test suite exactly.

Orbit — INTERACTIVE REANALYSIS

Orbit plans and runs analyses on Galaxy and builds the tool when none fits. It writes a UDT, has Galaxy validate it, runs it on the real data, reads any failure, fixes and re-creates it, and carries on with the analysis. Each definition is saved with the analysis notebook, keyed to the exact tool that ran.

notebook ↔ production software **personal code ↔ community tools** **an algorithm ↔ infrastructure to run it**

Acknowledgements: Work in the authors' groups is supported by National Institutes of Health grants U24HG002371, U24HG006620, U24HG010263, U24CA284167, U24AI183870, and R01GM151683, and by National Science Foundation grants 2419522 and 2138585. UDT jobs run on Jetstream2 at Indiana University (NSF OAC-2005506) through ACCESS (NSF grants 2138259, 2138286, 2138307, 2137603, and 2138296). Foldseek UDT on usegalaxy.org: Anton Nekrutenko. Related CSHL posters: Orbit (Cain) · Galaxy Workflow Foundry (Chilton) · HighGen (Burhans) · disassembler (Seacord) · VGP diploid curation (Larivière).

Request access · Docs · Source